Conexión

| | | | |
|---|---|---|---|
| llave-escalabilidad.pem | 19/04/2026 11:33 p. m. | Archivo PEM | 3 KB |

ssh -i llave-escalabilidad.pem scalability_lab@4.172.211.227

```
C:\Users\pacoa\Downloads>ssh -i llave-escalabilidad.pem scalabil
ity_lab@4.172.211.227
The authenticity of host '4.172.211.227 (4.172.211.227)' can't b
e established.
ED25519 key fingerprint is SHA256:fk3/XNdLCxsxtZzbsbvY0ofbuxZs99
XyVXQrZXy5j+I.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprin
t])? yes
```

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in t
he
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitte
d by
applicable law.

To run a command as administrator (user "root"), use "sudo <comm
and>".
See "man sudo_root" for details.

scalability_lab@VERTICAL-SCALABILITY:~$
```

```
PS C:\Users\pacoa> node -v
v22.20.0
```

```
Run `npm audit` for details.
PS C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp>  node FibonacciApp.js
Fibonacci App listening on port 3000!
```

cd Laboratorio-9-ARSW/FibonacciApp

```
-bash: cd: Laboratorio-9-ARSW/FibonacciApp: No such file or directory
scalability_lab@VERTICAL-SCALABILITY:~$ git clone https://github.com/D1052023/Laboratorio-9-ARSW
Cloning into 'Laboratorio-9-ARSW'...
remote: Enumerating objects: 39, done.
remote: Counting objects: 100% (39/39), done.
remote: Compressing objects: 100% (33/33), done.
remote: Total 39 (delta 3), reused 34 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (39/39), 3.55 MiB | 13.87 MiB/s, done.
Resolving deltas: 100% (3/3), done.
scalability_lab@VERTICAL-SCALABILITY:~/Laboratorio-9-ARSW/FibonacciApp$
```

```
scalability_lab@VERTICAL-SCALABILITY:~/Laboratorio-9-ARSW/FibonacciApp$ sudo apt install npm
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential bzip2 cpp cpp-13 cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu
  dpkg-dev eslint fakeroot fontconfig-config fonts-dejavu-core fonts-dejavu-mono g++ g++-13 g++-13-x86-64-linux-gnu
  g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu gyp handlebars javascript-common
  libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl libaom3 libasan8 libatomic1 libauthen-sasl-perl
  libbinutils libc-dev-bin libc-devtools libc6-dev libcares2 libcc1-0 libclone-perl libcrypt-dev libctf-nobfd0 libctf0
  libdata-dump-perl libde265-0 libdeflate0 libdpkg-perl libdrm-amdgpu1 libdrm-intel1 libegl-mesa0 libegl1 libencode-locale-perl
  libfakeroot libfile-basedir-perl libfile-desktopentry-perl libfile-fcntllock-perl libfile-listing-perl libfile-mimeinfo-perl
```

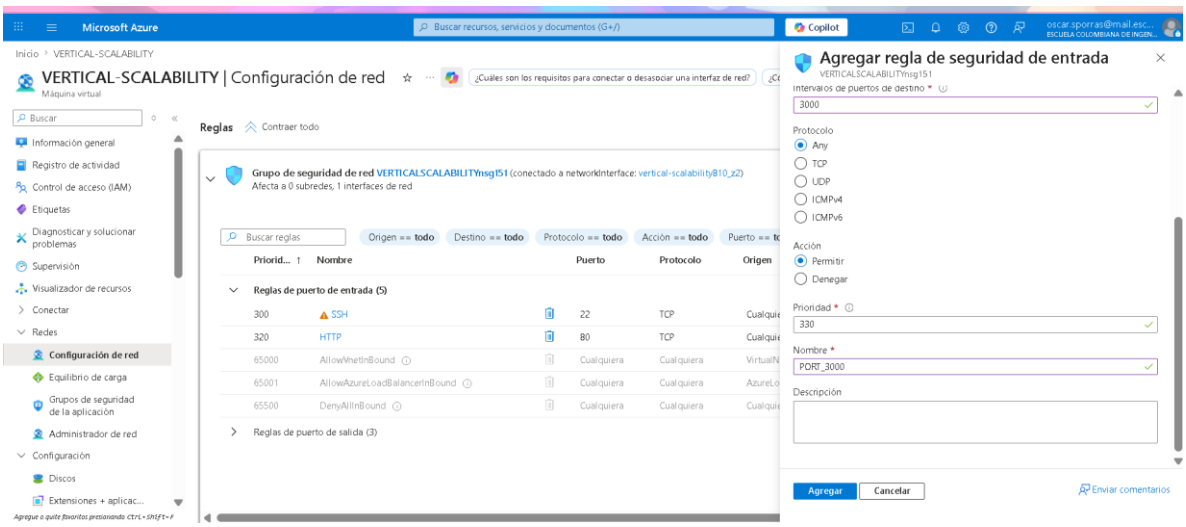
```
scalability_lab@VERTICAL-SCALABILITY:~/Laboratorio-9-ARSW/FibonacciApp$ npm install
npm WARN old lockfile
npm WARN old lockfile The package-lock.json file was created with an old version of npm,
npm WARN old lockfile so supplemental metadata must be fetched from the registry.
npm WARN old lockfile
npm WARN old lockfile This is a one-time fix-up, please be patient...
npm WARN old lockfile

added 51 packages, and audited 52 packages in 4s

7 vulnerabilities (3 low, 4 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
scalability_lab@VERTICAL-SCALABILITY:~/Laboratorio-9-ARSW/FibonacciApp$ node FibonacciApp.js
Fibonacci App listening on port 3000!
```



Reglas de puerto de entrada (6)

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 300 | SSH | | 22 | TCP | Cualquiera | Cualquiera | Allow |
| 320 | HTTP | | 80 | TCP | Cualquiera | Cualquiera | Allow |
| 330 | PORT_3000 | | 3000 | Cualquiera | Cualquiera | Cualquiera | Allow |
| 65000 | AllowVnetInBound | | Cualquiera | Cualquiera | VirtualNetwork | VirtualNetwork | Allow |
| 65001 | AllowAzureLoadBalancerInBound | | Cualquiera | Cualquiera | AzureLoadBalancer | Cualquiera | Allow |
| 65500 | DenyAllInBound | | Cualquiera | Cualquiera | Cualquiera | Cualquiera | Deny |

Reglas de puerto de salida (3)

No seguro 4.172.211.227:3000/fibonacci/6

The answer is 8

No seguro 4.172.211.227:3000/fibonacci/1000000

The answer is
195328212870775773163201494759625633244354299659187339695340519457162525788701569476664198763415014612887952433522023608462551091201956023374401543811519663615

No seguro 4.172.211.227:3000/fibonacci/1010000

The answer is
146949284679595242601574258743848413919764336681708419968365087254334695712876205283607354963739169817196094145602117765237670871227937939275137141167643209588

No seguro 4.172.211.227:3000/fibonacci/1020000

The answer is
110552858445138351515231425332219050625798548970902290993432297730943190110044033772175034125543151372035937203475522868753418076483090688578075775124589524022

No seguro 4.172.211.227:3000/fibonacci/1030000

The answer is
831711058481109052065537044556689912044091884970490850215269682455139024711114272158688785381865134226978570100817234959386980569408568008183300512907463218791

No seguro 4.172.211.227:3000/fibonacci/1040000

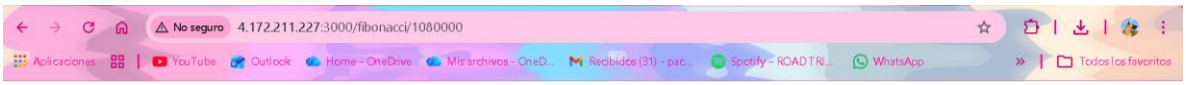
The answer is
625712708408207380957509511912655096469979914959956769350294973003039605938438684487875835526496906393766778499998566764671943730973410005935976866412324915644

No seguro 4.172.211.227:3000/fibonacci/1050000

The answer is
470736068098614810288686122743448117894063034777807063735473769722185444888474957158338913150383051702062628160801600363632937211262042968946252218664643726044

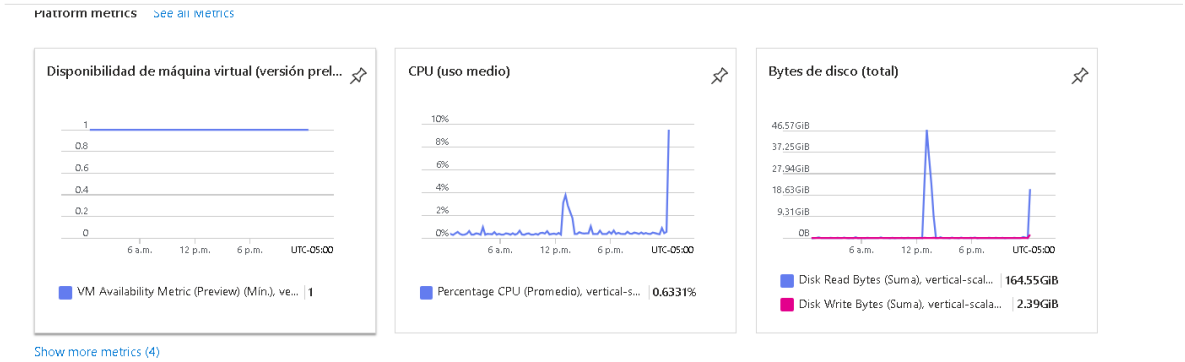
The answer is
3541440709629626004080256345633493970771539487596700096486349181454206399719089630306877128295126697547355428425785144880546515753782470417576613328441218249 41

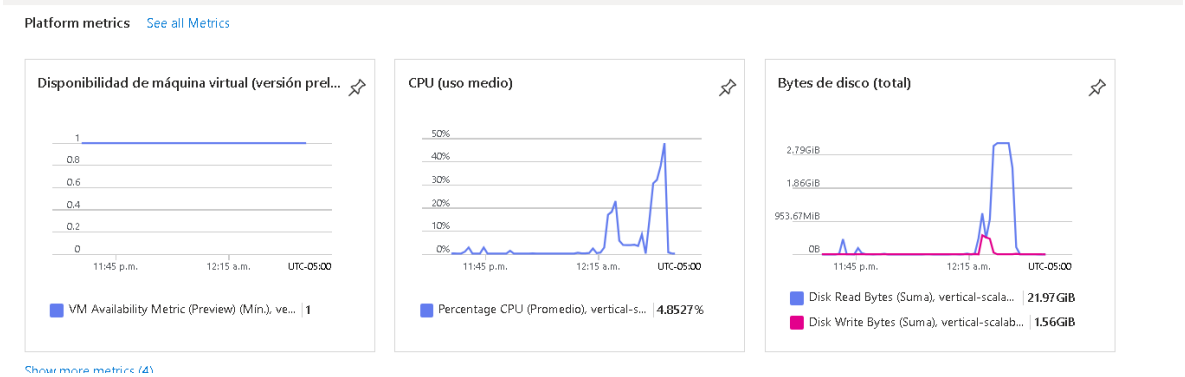
The answer is
2664296014214614747990651221646090798036020861184094719924022697064558365387616223838840489529297567196589384777743950026424591851084403919500002540410230915 4




The answer is
2004402680541349868174022816787004426653062499507891452908143794176865394569259093109007515191961089766625053731315998188028497602847432760851420199497161156 42

The answer is
1507951850817774765105016946873667968862954246631857635246247431496928021514429014122213864611327465493471485104652108049370635622819736729127845671645952334 72



```
{
    "id": "373cbe10-3dd7-4a9a-8519-bb96bfcdf4b5",
    "name": "[ARSW_LOAD-BALANCING_AZURE]",
    "values": [
        {
            "key": "VM1",
            "value": "4.172.211.227",
            "enabled": true
        },
        {
            "key": "nth",
            "value": "1000000",
            "enabled": true
        }
    ],
    "_postman_variable_scope": "environment",
    "_postman_exported_at": "2019-11-06T02:28:34.578Z",
    "_postman_exported_using": "Postman/7.9.0"
}
```


```
PS C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp\postman\part1> newman run ARSW_LOAD-BALANCING_AZURE.postman_collection.json -e [ARSW_LOAD-BALANCING_AZURE].postman_environment.jso
n -n 10
>>

→ fibonacci
  GET http://4.172.211.227:3000/fibonacci/1000000 [200 OK, 209.24kB, 9.6s]
```

| | executed | failed |
|---|---|---|
| iterations | 10 | 0 |
| requests | 10 | 0 |
| test-scripts | 10 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 1m 37s | | |
| total data received: 2.09MB (approx) | | |
| average response time: 9.6s [min: 9.3s, max: 9.8s, s.d.: 134ms] | | |

```
PS C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp\postman\part1>
```

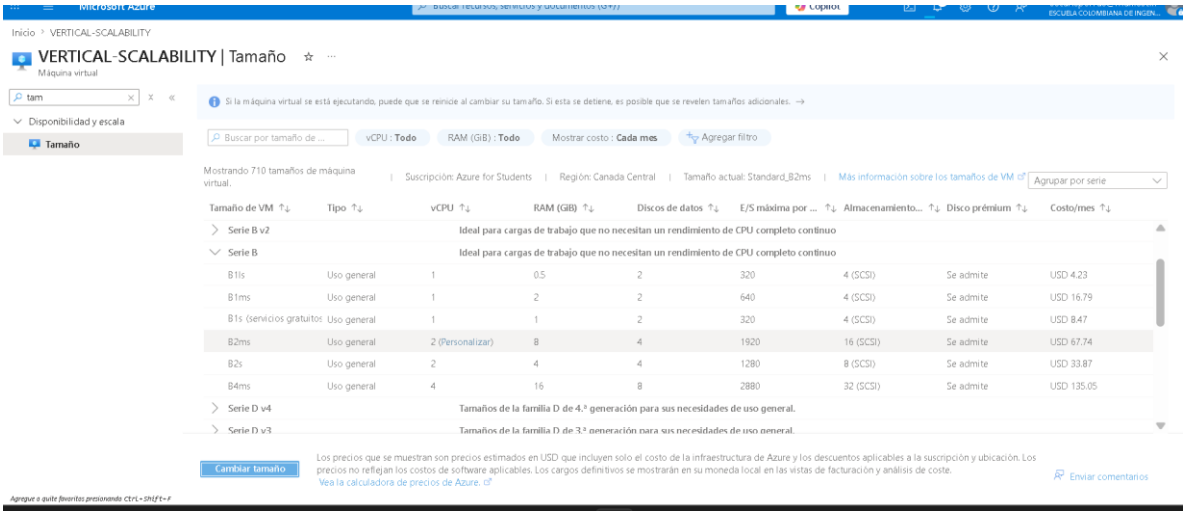
```
PS C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp\postman\part1> newman run ARSW_LOAD-BALANCING_AZURE.postman_collection.json -e [ARSW_LOAD-BALANCING_AZURE].postman_environment.j
n -n 10
>>

Iteration 10/10

→ fibonacci
  GET http://4.172.211.227:3000/fibonacci/1000000 [200 OK, 209.24kB, 9.5s]
```

| | executed | failed |
|---|---|---|
| iterations | 10 | 0 |
| requests | 10 | 0 |
| test-scripts | 10 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 1m 34.9s | | |
| total data received: 2.09MB (approx) | | |
| average response time: 9.4s [min: 9.2s, max: 9.6s, s.d.: 110ms] | | |

```
PS C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp\postman\part1>
```

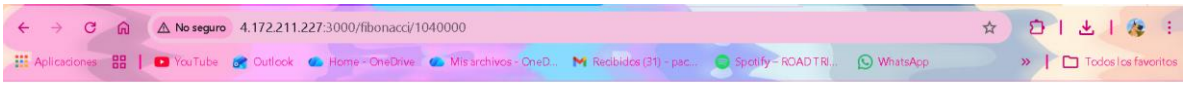


The answer is
195328212870775773163201494759625633244354299659187339695340519457162525788701569476664198763415014612887952433522023608462551091201956023374401543811519663615

The answer is
146949284679595242601574258743848413919764336681708419968365087254334695712876205283607354963739169817196094145602117765237670871227937939275137141167643209588

The answer is
110552858445138351515231425332219050625798548970902290993432297730943190110044033772175034125543151372035937203475522868753418076483090688578075775124589524022

The answer is
831711058481109052065537044556689912044091884970490850215269682455139024711114272158688785381865134226978570100817234959386980569408568008183300512907463218791




The answer is
625712708408207380957509511912655096469979914959956769350294973003039605938438684487875835526496906393766778499998566764671943730973410005935976866412324915644

The answer is
470736068098614810288686122743448117894063034777807063735473769722185444888474957158338913150383051702062628160801600363632937211262042968946252218664643726044

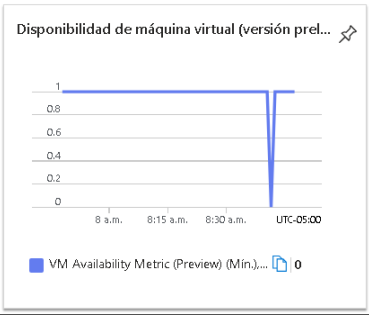
The answer is
354144070962962600408025634563349397077153948759670009648634918145420639971908963030687712829512669754735542842578514488054651575378247041757661332844121824 94

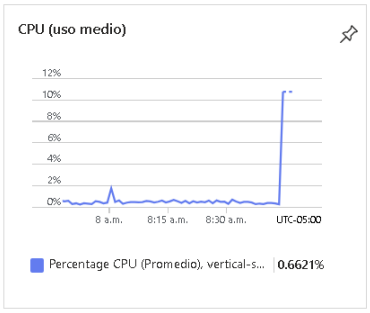
The answer is
2664296014214614747990651221646090798036020861184094719924022697064558365387616223838840489529529756719658938477774395002642459185108440391950000254041023091 54

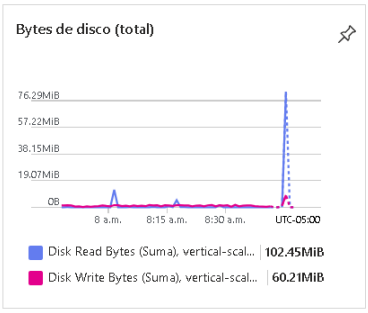
The answer is
2004402680541349868174022816787004426653062499507891452908143794176865394569259093109007515191961089766625053731315998188028497602847432760851420199497161156 42

The answer is
1507951850817774765105016946873667968862954246631857635246247431496928021514429014122213864611327465493471485104652108049370635622819736729127845671645952334 72

Platform metrics See all Metrics



```
→ fibonacci
  GET http://4.172.211.227:3000/fibonacci/1000000 [200 OK, 209.24kB, 8.7s]
```

| | executed | failed |
|---|---|---|
| iterations | 10 | 0 |
| requests | 10 | 0 |
| test-scripts | 10 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 1m 29.4s | | |
| total data received: 2.09MB (approx) | | |
| average response time: 8.8s [min: 8.6s, max: 9.1s, s.d.: 143ms] | | |

```
PS C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp\postman\part1>
```

**Comparativa de Resultados: Antes vs. Después**

**1. Rendimiento y Tiempo de Respuesta**

- **Antes (Máquina Inicial):** El tiempo de respuesta promedio fue de **9.4 segundos**.
- **Después (Máquina B2ms):** El tiempo de respuesta promedio bajó a **8.8 segundos**.
- **Análisis:** Se observa una mejora en la velocidad de procesamiento del **6.4%** aproximadamente. Aunque hay una mejora, no es drástica.

**2. Capacidad de Procesamiento y Concurrencia**

- **Antes:** La duración total de la ejecución de 10 iteraciones fue de **1m 34.9s**.
- **Después:** La duración total bajó a **1m 29.4s**.
- **Análisis:** La máquina con más núcleos pudo finalizar el lote de peticiones concurrentes en menos tiempo, reduciendo la latencia acumulada.

**3. Comportamiento de la CPU**

- **Antes:** Las métricas muestran picos de CPU que rozan el **50%** de uso medio en periodos cortos de ráfaga.

- **Después:** Se observa una mayor estabilidad en la disponibilidad, aunque el consumo de recursos sigue siendo alto debido a que el cálculo de Fibonacci es computacionalmente intensivo.

**Evaluación del Requerimiento No Funcional (Escalabilidad)**

**¿Logramos cumplir el escenario de calidad?**

**Conclusión: Parcialmente.**

1. **Efectividad del Escalamiento Vertical:** Al aumentar los recursos (CPUs y RAM), el sistema mostró una mejora en los tiempos de respuesta y mayor estabilidad ante peticiones concurrentes. Sin embargo, la mejora no es proporcional al aumento de costo y hardware.

2. **Limitación Tecnológica:** La aplicación está desarrollada en **Node.js**, que es *single-threaded* (un solo hilo). Al realizar cálculos matemáticos pesados (Fibonacci), un núcleo se satura bloqueando a los demás. Por mucho que aumentemos la CPU (Escalamiento Vertical), el hilo principal sigue siendo el cuello de botella.

3. **Veredicto:** Para este tipo de cargas de trabajo, el **Escalamiento Vertical** tiene un techo muy bajo. Para cumplir realmente con una escalabilidad alta, se recomendaría un **Escalamiento Horizontal** (varias máquinas pequeñas con un balanceador de carga) para distribuir el procesamiento de los cálculos.

Mostrando 710 tamaños de máquina virtual. | Suscripción: Azure for Students | Región: Canada Central | Tamaño actual: Standard_B1ls | Más información sobre los tamaños de VM | Agrupar por serie

| Tamaño de VM | Tipo | vCPU | RAM (GiB) | Discos de datos | E/S máxima por ... | Almacenamiento... | Disco prémium | Costo/mes |
|---|---|---|---|---|---|---|---|---|
| Más usados por los usuarios de Azure | | Los tamaños más usados por los usuarios en Azure. | | | | | | |
| B2as_v2 | Uso general | 2 | 8 | 4 | 3750 | N/D | Se admite | USD 61.03 |
| B4as_v2 | Uso general | 4 | 16 | 8 | 6400 | N/D | Se admite | USD 121.91 |
| Serie B v2 | | Ideal para cargas de trabajo que no necesitan un rendimiento de CPU completo continuo | | | | | | |
| Serie B | | Ideal para cargas de trabajo que no necesitan un rendimiento de CPU completo continuo | | | | | | |
| B1ls | Uso general | 1 (Personalizar) | 0.5 | 2 | 320 | 4 (SCSI) | Se admite | USD 4.23 |
| B1ms | Uso general | 1 | 2 | 2 | 640 | 4 (SCSI) | Se admite | USD 16.79 |
| B1s (servicios gratuitos | Uso general | 1 | 1 | 2 | 320 | 4 (SCSI) | Se admite | USD 8.47 |
| B2ms | Uso general | 2 | 8 | 4 | 1920 | 16 (SCSI) | Se admite | USD 67.74 |
| B2s | Uso general | 2 | 4 | 4 | 1280 | 8 (SCSI) | Se admite | USD 33.87 |

Los precios que se muestran son precios estimados en USD que incluyen solo el costo de la infraestructura de Azure y los descuentos aplicables a la suscripción y ubicación. Los

## 1. Recursos creados junto con la VM

Cuando creas una máquina virtual en Azure de forma estándar, se crea un **Grupo de Recursos** que contiene generalmente **6 recursos** principales:

1. **Virtual Machine (VM):** El recurso de cómputo principal.

2. **Network Interface (NIC):** El adaptador de red que permite la comunicación.

3. **Network Security Group (NSG):** El firewall que define las reglas de tráfico.
4. **Public IP Address:** La dirección para acceder desde internet.
5. **Virtual Network (VNet):** La red privada lógica donde reside la VM.
6. **Disk:** El almacenamiento persistente (OS Disk).

**2. Descripción de funciones**

- **Virtual Machine:** Proporciona los núcleos de CPU y RAM para ejecutar el código.
- **Network Interface:** Conecta la VM con la red virtual y la IP pública.
- **Network Security Group:** Filtra el tráfico entrante y saliente mediante reglas de seguridad.
- **Public IP:** Permite que Newman o tu navegador encuentren la máquina en la red global.
- **Virtual Network:** Aísla tu infraestructura de otros usuarios en Azure.
- **Disk:** Guarda el sistema operativo (Ubuntu) y los archivos del proyecto (FibonacciApp.js).

**3. Conexión SSH y Reglas de Puerto**

- **¿Por qué se cae la App al cerrar SSH?:** Al ejecutar node FibonacciApp.js directamente, el proceso se asocia a la sesión de tu terminal (TTY). Si cierras la conexión, el sistema operativo envía una señal SIGHUP que termina todos los procesos hijos de esa sesión.
- **Inbound Port Rule:** Por defecto, Azure bloquea todo el tráfico entrante por seguridad. Debes abrir explícitamente el puerto **3000** para que las peticiones HTTP de Newman puedan llegar al servidor Node.js.

**4. Tabla de Tiempos e Interpretación**

Basado en tus ejecuciones:

| Petición | Tiempo (B1ls/Original) | Tiempo (B2ms/Escalado) |
|---|---|---|
| **Promedio** | **9.4s** | **8.8s** |
| **Mínimo** | 9.2s | 8.6s |

| Petición | Tiempo (B1ls/Original) | Tiempo (B2ms/Escalado) |
|---|---|---|
| **Máximo** | 9.6s | 9.1s |

**Interpretación:** La función tarda tanto porque el algoritmo de Fibonacci implementado es **recursivo o lineal sin memoización**, lo que implica una complejidad computacional alta. Además, al ser un cálculo intensivo de CPU, el tiempo depende directamente de la velocidad de ciclo del procesador.

**5. Consumo de CPU**

**[Cargar Imagen image_147e83.png]**

**Interpretación:** La gráfica muestra picos que llegan al **50% - 100%** de la capacidad de la instancia. Esto ocurre porque Node.js es *single-threaded*; el cálculo de Fibonacci "secuestra" el único hilo disponible, poniendo el núcleo asignado al máximo esfuerzo hasta terminar el cálculo.

**6. Resumen de Postman e Interpretación**

**[Cargar Imagen image_147ea1.png]**

- **Interpretación:** Las peticiones son consistentes (desviación estándar de ~110ms). Esto indica que, aunque el servidor es lento, el tiempo de procesamiento es predecible para el mismo valor de entrada.
- **Fallos:** El fallo inicial ENOTFOUND se debió a una mala configuración de la URL en el archivo de ambiente (repetía el protocolo http://).

**7. Diferencia entre B1ls y B2ms**

- **B1ls:** Es una instancia de bajo costo (1 vCPU, 0.5 GB RAM). Usa "créditos de CPU", lo que significa que si se esfuerza mucho por mucho tiempo, Azure la limita (throttling).
- **B2ms:** Es una instancia de rendimiento estándar (2 vCPUs, 8 GB RAM). Permite ráfagas de CPU más largas y tiene una base de rendimiento mucho más alta y estable para multitarea.

**8. ¿Es el escalamiento vertical una buena solución?**

- **Análisis:** No es la mejor solución. Aunque los tiempos bajaron de **9.4s a 8.8s**, la mejora es mínima comparada con el aumento de recursos.

- **¿Qué pasa con la App?:** La aplicación se detiene. Al cambiar el tamaño, Azure apaga la VM, mueve los discos a un nuevo hardware físico con más capacidades y la vuelve a encender. Debes reiniciar el servicio manualmente tras el cambio.

**9. Efectos en la infraestructura y Negativos**

- **Efectos:** Azure desasocia la VM de su nodo actual y la migra.
- **Negativos:** 1. **Downtime:** El servicio queda fuera de línea durante varios minutos.

2. **Costo:** El precio de la B2ms es significativamente mayor.

3. **Límite físico:** Eventualmente no podrás escalar más si llegas al tamaño máximo de la región.

**10. ¿Hubo mejora?**

- **Respuesta: Sí**, hubo una mejora ligera (~0.6s de diferencia).
- **¿Por qué?:** Porque la serie B2ms tiene procesadores más modernos y mayor prioridad de cómputo. Sin embargo, la mejora no fue mayor porque Node.js no puede usar la segunda CPU adicional para un solo cálculo de Fibonacci debido a su naturaleza de un solo hilo.

**11. Ejecuciones Paralelas (Newman x4)**

- **Comportamiento:** El sistema **empeora porcentualmente**.
- **Explicación:** Al lanzar 4 ejecuciones, el tiempo de respuesta no se mantiene en 5s, sino que podría duplicarse o triplicarse. Al tener solo 1 o 2 núcleos para procesar 4 cálculos pesados simultáneos, las peticiones entran en una cola de espera, aumentando drásticamente la latencia de las últimas peticiones en entrar.

The answer is 3

**Parte 2 - Escalabilidad horizontal**

Inicio

# Crear equilibrador de carga ...

IP públicas, o bien internos, a los que solo se puede acceder desde una red virtual. Los equilibradores de carga de Azure también son compatibles con la traducción de direcciones de red (NAT) para enrutar el tráfico entre las direcciones IP públicas y privadas. Más información.

**Detalles del proyecto**

Suscripción * Azure for Students

Grupo de recursos * SCALABILITY_LAB

Crear nuevo

**Detalles de instancia**

Nombre * SCALABILITY_LAB_LOAD_BALANCE

Región * Canada Central

SKU *
- (•) Estándar (distribuye el tráfico a los recursos de back-end)
- ( ) Puerta de enlace (tráfico directo a aplicaciones virtuales de red)

Tipo *
- (•) Pública
- ( ) Interno

Nivel *
- (•) Regional
- ( ) Global

Revisar y crear | < Anterior | Siguiente: Configuración de IP de front-end > | Descargar una plantilla para la automatización | Enviar comentarios

Inicio

# Crear equilibrador de carga

Datos básicos | **Configuración de IP de front-end** | Grupos de back-end | Reglas de entrada

Una configuración IP de front-end es una dirección IP que se usa para la comunicación entrante o saliente, como se define en equilibrio de carga, NAT entrante y reglas de salida.

+ Agregar una configuración de IP de front-end

| Nombre | Dirección IP |
|---|---|
| LB-Frontend-IP | (nuevo) Public-IP-LB (Por crearse) |

Revisar y crear | < Anterior | Siguiente: Grupos de back-end > | Descargar una plantilla para la automatización

Enviar comentarios

Inicio > CreateLoadBalancerBladeV2-20260420091209 | Información general > SCALABILITY_LAB_LOAD_BALANCE

## SCALABILITY_LAB_LOAD_BALANCE | Grupos de back-end

Equilibrador de carga

back

Configuración

Grupos de back-end

+ Agregar | Actualizar | Exportar a CSV

El grupo de back-end es un componente crítico del equilibrador de carga. El grupo de back-end define el grupo de recursos que atenderá el tráfico para una regla de equilibrio de carga determinada. Más información.

Agregar filtro

| Grupo de backend | Nombre del recurso | Dirección IP | Interfaz de red | Zona de disponibilidad | Recuento de reglas | Estado del recurso | Estado de administra- |
|---|---|---|---|---|---|---|---|
| SCALABILITY_LAB_BACKEND_POOL (1) | | | | | | | |
| SCALABILITY_LAB_BACKENE | - | - | - | - | 0 | - | - |

Implementación correcta

La implementación 'RegionalLoadBalancerBackendPoolCreateOrUpdate-20260420092400-86' en el grupo de recursos 'SCALABILITY_LAB' se realizó correctamente.

Inicio > CreateLoadBalancerBladeV2-20260420091209 | Información general > SCALABILITY_LAB_LOAD_BALANCE

## SCALABILITY_LAB_LOAD_BALANCE | Sondeos de estado

Equilibrador de carga

Buscar

- Información general
- Registro de actividad
- Control de acceso (IAM)
- Etiquetas
- Diagnosticar y solucionar problemas
- Visualizador de recursos
- Configuración
  - Configuración de IP de front-end
  - Grupos de back-end
  - Sondeos de estado
  - Reglas de equilibrio de carga

Agregar | Actualizar | Enviar comentarios

Escriba para filtrar...

Para comprobar el estado de mantenimiento de las instancias, vaya a la página Reglas de equilibrio de carga

| Nombre | Protocolo | Puerto | Ruta | Usado por |
|---|---|---|---|---|
| SCALABILIT... | Tcp | 3000 | - | - |

Inicio > CreateLoadBalancerBladeV2-20260420091209 | Información general > SCALABILITY_LAB_LOAD_BALANCE

## SCALABILITY_LAB_LOAD_BALANCE | Reglas de equilibrio de carga

Equilibrador de carga

Buscar

- Información general
- Registro de actividad
- Control de acceso (IAM)
- Etiquetas
- Diagnosticar y solucionar problemas
- Visualizador de recursos
- Configuración
  - Configuración de IP de front-end
  - Grupos de back-end
  - Sondeos de estado

Agregar | Actualizar | Exportar a CSV | Eliminar

Se usa una regla de equilibrador de carga para definir cómo se distribuye el tráfico entrante a todas las instancias del grupo de back-end. Una regla de equilibrio de carga asigna una configuración IP de front-end y un puerto determinados a varios puertos y direcciones IP de back-end. Un ejemplo sería una regla creada en el puerto 80 para equilibrar la carga del tráfico web. Obtenga más información.

Filtrar por nombre...

| Nombre | Protocolo | Grupo de back-end | Sondeo de estado | Estado de mantenimiento |
|---|---|---|---|---|
| SCALABILITY_LAB_LOAD_BALANCING_RULE | TCP/80 a TCP/3000 | SCALABILITY_LAB_BACKEND_POOL | SCALABILITY_LAB_HEALTH_PROBE | Ver detalles |

### Detalles del proyecto

Seleccione la suscripción para administrar recursos implementados y los costes. Use los grupos de recursos como carpetas para organizar y administrar todos los recursos.

Suscripción * — Azure for Students

Grupo de recursos * — SCALABILITY_LAB

Crear nuevo

### Detalles de instancia

Nombre de máquina virtual * — VM1

Región * — (Canada) Canada Central

Implementación en una zona extendida de Azure

Opciones de disponibilidad — Zona de disponibilidad

Opciones de zona

- (●) Zona autoseleccionada
  Elija hasta 3 zonas de disponibilidad, una máquina virtual por zona
- ( ) Zona seleccionada por Azure (versión preliminar)
  Permitir que Azure asigne la mejor zona para sus necesidades

Zona de disponibilidad * — Zona 2

Ahora puede seleccionar varias zonas. Si selecciona varias zonas, se creará una VM por zona. Más información

< Anterior | Siguiente: Discos > | Revisar y crear

Nombre de usuario * vm1

Origen de clave pública SSH: Usar la clave pública existente

Los formatos Ed25519 y RSA SSH son compatibles con la imagen de VM seleccionada. Ed25519 proporciona un nivel de seguridad fijo de no más de 128 bits para claves de 256 bits, mientras que RSA podría ofrecer una mejor seguridad con claves de más de 3072 bits.

Clave pública SSH *

```
ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIMLv79C+LWTrCqdGAhUGFlr+gs5Pl99X8AwiHwst
1rvl oscar.sporras@mail.escuelaing.edu.co
```

Más información sobre la creación y el uso de claves SSH en Azure

< Anterior | Siguiente: Discos > | Revisar y crear

**Equilibrio de carga**

Puede colocar esta máquina virtual en el grupo de back-end de una solución de equilibrio de carga de Azure existente.
Más información

Opciones de equilibrio de carga
- Ninguno
- Azure Load Balancer
  Admite todo el tráfico de red TCP/UDP, el reenvío de puertos y los flujos de salida.
- Puerta de enlace de aplicaciones
  Equilibrador de carga de tráfico web para HTTP/HTTPS con enrutamiento basado en URL, finalización de SSL, persistencia de sesión y firewall de aplicaciones web.

Seleccionar un equilibrador de carga * SCALABILITY_LAB_LOAD_BALANCE
Crear un equilibrador de carga

Seleccionar un grupo de back-end * SCALABILITY_LAB_BACKEND_POOL
Crear nuevo

carga ya existente. Más información

**Interfaz de red**

Al crear una máquina virtual, se crea una interfaz de red automáticamente.

Red virtual * SCALABILITY_LAB_VIRTUAL_NETWORK
Crear nuevo

Subred * SCALABILITY_LAB_SUBNET (10.1.0.0/24)
Administrar configuración de subred

IP pública (nuevo) VM1-ip
Crear nuevo

Las direcciones IP públicas tienen un coste nominal. Precio estimado

Grupo de seguridad de red de NIC
- Ninguno
- Básico
- Opciones avanzadas

Configurar el grupo de seguridad de red * LOAD_BALANCER_NSG
Crear nuevo

Actualice los puertos del grupo de seguridad de red para permitir el tráfico del recurso Azure Load Balancer. Haz clic aquí para actualizar los puertos

ssh -i C:\Users\pacoa\.ssh\id_ed25519 VM1@20.151.242.66

ssh -i C:\Users\pacoa\.ssh\id_ed25519 VM2@20.175.134.16

```
git clone https://github.com/D1052023/Laboratorio-9-ARSW.git

curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash

source ~/.bashrc

nvm install node

cd Laboratorio-9-ARSW/FibonacciApp

sudo apt-get update && sudo apt-get install libatomic1 -y

npm install

npm install forever -g

forever start FibonacciApp.js

forever list
```

MQ3 y 4

```
sudo apt-get update

sudo apt-get install libatomic1 -y
```

VM1

```
tar -czvf proyecto.tar.gz Laboratorio-9-ARSW/

scp proyecto.tar.gz VM3@10.1.0.8:~/

cp /home/VM1/.nvm/versions/node/v25.9.0/bin/node ~/node-bin

scp ~/node-bin VM3@10.1.0.8:~/

find /usr/lib -name "libatomic.so.1" 2>/dev/null

cp /usr/lib/x86_64-linux-gnu/libatomic.so.1 ~/


tar -xzvf proyecto.tar.gz

chmod +x ~/node-bin

cd Laboratorio-9-ARSW/FibonacciApp

mv ~/libatomic.so.1 ~/Laboratorio-9-ARSW/FibonacciApp/
```

cd ~/Laboratorio-9-ARSW/FibonacciApp/

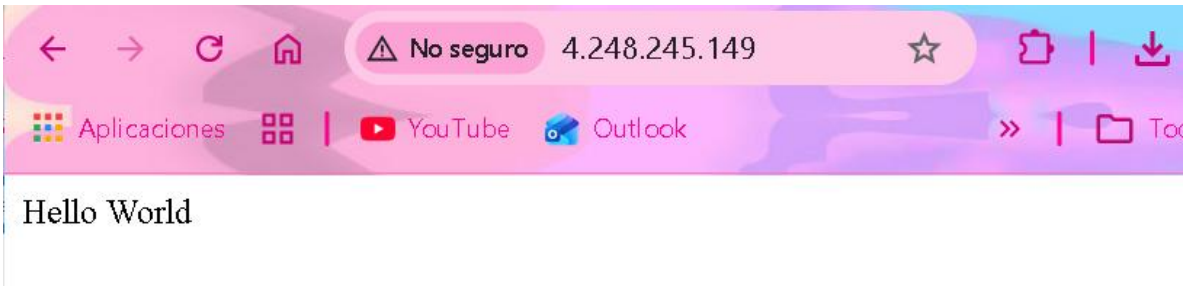
LD_LIBRARY_PATH=. nohup ~/node-bin FibonacciApp.js &

ps aux | grep node-bin

```
VM1@VM1:~/Laboratorio-9-ARSW/FibonacciApp$ forever start Fibonac
ciApp.js
warn:    --minUptime not set. Defaulting to: 1000ms
warn:    --spinSleepTime not set. Your script will exit if it do
es not stay up for at least 1000ms
info:    Forever processing file: FibonacciApp.js
(node:8734) Warning: Accessing non-existent property 'padLevels'
 of module exports inside circular dependency
(Use `node --trace-warnings ...` to show where the warning was c
reated)
(node:8734) Warning: Accessing non-existent property 'padLevels'
 of module exports inside circular dependency
(node:8734) [DEP0060] DeprecationWarning: The `util._extend` API
 is deprecated. Please use Object.assign() instead.
```

```
is deprecated. Please use Object.assign() instead.
nfo:    Forever processes running
ata:        uid  command
cript          forever pid  id logfile                    upti
e
ata:    [0] yhRa /home/VM2/.nvm/versions/node/v25.9.0/bin/node
ibonacciApp.js 5913    5921    /home/VM2/.forever/yhRa.log 0:0:
5:3.2110000000000127
ata:    [1] 9T9Q /home/VM2/.nvm/versions/node/v25.9.0/bin/node
ibonacciApp.js 5976    6012    /home/VM2/.forever/9T9Q.log STOP
ED
M2@VM2:~/Laboratorio-9-ARSW/FibonacciApp$ forever list
```

```
nohup: VM3@VM3:~/Laboratorio-9-ARSW/FibonacciApp$ ignoring input
and appending output to 'nohup.out'
VM3@VM3:~/Laboratorio-9-ARSW/FibonacciApp$ps aux | grep node-binn
VM3         3314  0.1 13.2 1411440 52956 ttyS0    Sl   01:00   0:0
0 /home/VM3/node-bin FibonacciApp.js
VM3         3355  0.0  0.5   7088  2232 ttyS0    S+   01:03   0:0
0 grep --color=auto node-bin
VM3@VM3:~/Laboratorio-9-ARSW/FibonacciApp$
```



The answer is 1


```
{
    "id": "373cbe10-3dd7-4a9a-8519-bb96bfcdf4b5",
    "name": "[ARSW_LOAD-BALANCING_AZURE]",
    "values": [
        {
            "key": "loadbalancer",
            "value": "4.248.245.149",
            "enabled": true
        },
        {
            "key": "nth",
            "value": "1000000",
```

```
→ fibonacci
  GET http://4.248.245.149/fibonacci/1000000 [200 OK, 209.24kB, 12.8s]
```

| | executed | failed |
|---|---|---|
| iterations | 10 | 0 |
| requests | 10 | 0 |
| test-scripts | 10 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 2m 8.9s | | |
| total data received: 2.09MB (approx) | | |
| average response time: 12.8s [min: 12.7s, max: 12.8s, s.d.: 55ms] | | |

```
PS C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp\postman\part2>
```

```
→ fibonacci
  GET http://4.248.245.149/fibonacci/1000000 [200 OK, 209.24kB, 11.1s]
```

| | executed | failed |
|---|---|---|
| iterations | 10 | 0 |
| requests | 10 | 0 |
| test-scripts | 10 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 1m 54.1s | | |
| total data received: 2.09MB (approx) | | |
| average response time: 11.3s [min: 10.9s, max: 12.1s, s.d.: 338ms] | | |

```
[sudo] password for VM4:
Sorry, try again.
[sudo] password for VM4:
Sorry, try again.
[sudo] password for VM4:
3000/tcp:              3859                                                    /
VM4@VM4:~/Laboratorio-9-ARSW/FibonacciApp$ cd ~/Laboratorio-9-ARSW/FibonacciApp/
VM4@VM4:~/Laboratorio-9-ARSW/FibonacciApp$ cd ~/Laboratorio-9-ARSW/FibonacciApp/
VM4@VM4:~/LaboratorLD_LIBRARY_PATH=. nohup ~/node-bin FibonacciApp.js > salida.log 2>&1 &p.js > salida.l
[1] 3953&
VM4@VM4:~/Laborator                          ps aux | grep node-bin
VM4         3953  0.8 15.5 1411440 62128 ttyS0    Sl   02:01   0:00 /home/VM4/node-bin FibonacciApp.js
VM4         3961  0.0  0.5   7088  2228 ttyS0    S+   02:01   0:00 grep --color=auto node-bin
VM4@VM4:~/Laboratorio-9-ARSW/FibonacciApp$
```

```
teration 10/10

→ fibonacci
  GET http://4.248.245.149/fibonacci/1000000 [200 OK, 209.24kB, 14.3s]
```

| | executed | failed |
|---|---|---|
| iterations | 10 | 0 |
| requests | 10 | 0 |
| test-scripts | 10 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |

total run duration: 4m 6.8s

total data received: 2.09MB (approx)

average response time: 24.6s [min: 12.6s, max: 1m 58.9s, s.d.: 31.4s]

```
S C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp\postman\part2>
```

```
→ fibonacci
  GET http://4.248.245.149/fibonacci/1000000 [200 OK, 209.24kB, 13.3s]
```

| | executed | failed |
|---|---|---|
| iterations | 10 | 0 |
| requests | 10 | 0 |
| test-scripts | 10 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |

total run duration: 2m 15.4s

total data received: 2.09MB (approx)

average response time: 13.4s [min: 12.9s, max: 16.2s, s.d.: 936ms]

```
PS C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp\postman\part2>
```

```
→ fibonacci
  GET http://4.248.245.149/fibonacci/1000000 [200 OK, 209.24kB, 13.3s]
```

| | executed | failed |
|---|---|---|
| iterations | 10 | 0 |
| requests | 10 | 0 |
| test-scripts | 10 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 2m 8.9s | | |
| total data received: 2.09MB (approx) | | |
| average response time: 12.8s [min: 12.6s, max: 13.3s, s.d.: 188ms] | | |

```
PS C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp\postman\part2>
```

```
→ fibonacci
  GET http://4.248.245.149/fibonacci/1000000 [200 OK, 209.24kB, 12.8s]
```

| | executed | failed |
|---|---|---|
| iterations | 10 | 0 |
| requests | 10 | 0 |
| test-scripts | 10 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 0 | 0 |
| total run duration: 2m 8.4s | | |
| total data received: 2.09MB (approx) | | |
| average response time: 12.7s [min: 12.6s, max: 12.9s, s.d.: 67ms] | | |

```
PS C:\Users\pacoa\Desktop\Laboratorio-9-ARSW\FibonacciApp\postman\part2>
```

### 1. Balanceadores de Carga en Azure

Azure ofrece cuatro tipos principales de balanceadores según la capa del modelo OSI y el alcance:

- **Azure Load Balancer:** Trabaja en la **Capa 4 (TCP/UDP)**. Es ideal para tráfico no HTTP y baja latencia.

- **Application Gateway:** Trabaja en la **Capa 7 (HTTP/HTTPS)**. Permite tomar decisiones basadas en rutas de URL y terminación SSL.
- **Traffic Manager:** Balanceador basado en **DNS**. Distribuye tráfico a nivel global entre diferentes regiones.
- **Azure Front Door:** Red de entrega de contenido (CDN) con balanceo global en la **Capa 7**.

**2. SKU (Stock Keeping Unit)**

El SKU define las capacidades, rendimiento y costo de un recurso. Para los balanceadores existen:

- **Basic:** Gratuito, pero limitado. No soporta Availability Zones y el Backend Pool está limitado a un conjunto de disponibilidad.
- **Standard:** De pago. Ofrece seguridad por defecto (requiere NSG), soporta **Availability Zones** y permite hasta 1000 instancias en el Backend Pool. **Es el que usamos en este lab.**

**3. ¿Por qué el balanceador necesita una IP pública?**

Actúa como el **punto de entrada único** (VIP - Virtual IP) para los clientes externos. Sin ella, el tráfico de internet no sabría cómo llegar a tu red privada. El balanceador recibe la petición en la IP pública y realiza una traslación hacia las IPs privadas de tus VMs.

**4. Propósito de los componentes**

- **Backend Pool:** Es el grupo de recursos (VMs) que efectivamente procesarán las solicitudes. Define a quiénes se les puede enviar trabajo.
- **Health Probe (Sondeo de estado):** Es el "vigilante". Envía señales constantes a las VMs para verificar si la aplicación (en tu caso, el puerto 3000) responde. Si una VM falla, el balanceador deja de enviarle tráfico automáticamente.
- **Load Balancing Rule:** Define el "cómo". Vincula la IP frontal con el Backend Pool y el Health Probe, especificando el puerto y el protocolo.

**5. Sesión Persistente (Session Affinity)**

Determina si un cliente debe ser atendido siempre por la misma VM.

- **None (5-tuple):** El tráfico se reparte de forma puramente estadística.

- **Client IP (2-tuple):** El mismo cliente va a la misma VM.
- **Client IP and Protocol (3-tuple):** Igual al anterior, pero separa por protocolo.
- **Importancia:** Es vital para apps que guardan estado local (sesiones). **Afecta la escalabilidad** porque puede generar "hot spots" (una VM muy cargada mientras otras están libres).

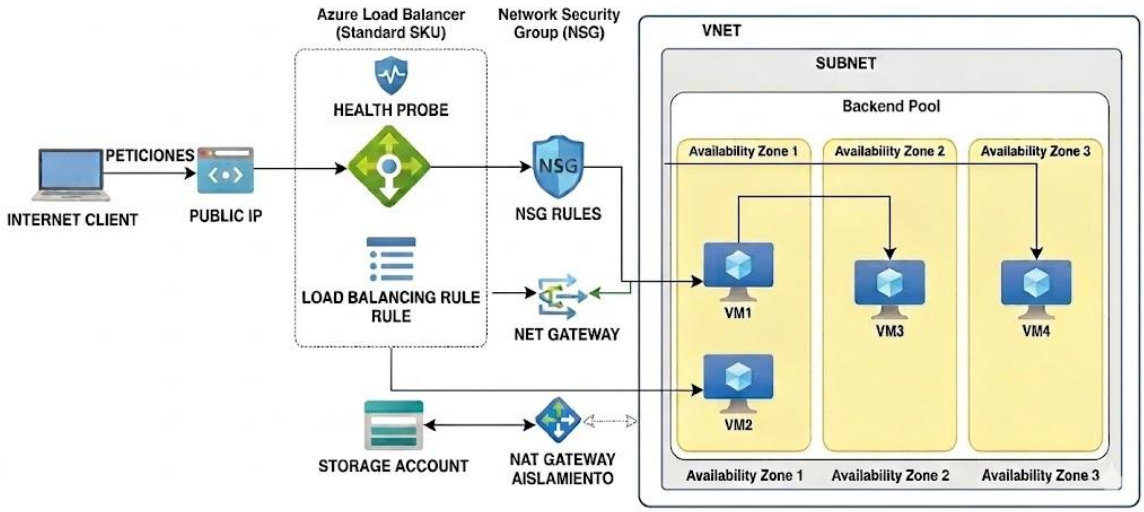
**6. Redes en Azure**

- **Virtual Network (VNet):** Tu red privada aislada en la nube.
- **Subnet:** División lógica de la VNet para organizar recursos (ej. subred de apps, subred de BD).
- **Address Space:** El rango total de IPs de la VNet (ej. 10.1.0.0/16).
- **Address Range:** El rango específico asignado a una Subnet (ej. 10.1.0.0/24).

**7. Availability Zones e IP Zone-Redundant**

- **Availability Zone:** Centros de datos físicamente separados dentro de una región. Seleccionamos 3 para garantizar que si un edificio se inunda o falla la energía, los otros dos sigan activos (**Alta Disponibilidad**).
- **Zone-redundant IP:** Significa que la dirección IP pública es servida por la infraestructura de todas las zonas de la región simultáneamente.

**8. Network Security Group (NSG)**

Es un **firewall lógico**. Contiene reglas de seguridad que permiten o deniegan el tráfico entrante/saliente hacia las VMs. Fue lo que configuraste para permitir el tráfico en el puerto 3000.

# DIAGRAMA DE DESPLIEGUE